

Intermediate (apply the follow-up template yourself) — Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic** (the plain answer a nervous first-timer would give), **Intermediate** (a properly structured senior-level answer, using this document's own Four-Depth Format), **Advanced** (a tough pushback on your own fix, or a harder variant of the scenario). Read Basic first even if you're confident — it's the fastest way to see exactly what each deeper level adds.

- [Hint 1 — The scenario, and why the first answer isn't the point](#)
- [Hint 2 — The follow-up plan, worked through](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The scenario, and why the first answer isn't the point

Basic Version

Scenario question for this exercise: *"Your RAG agent's retriever is returning irrelevant chunks in production, but it looked fine in testing. Walk me through how you'd debug it."*

Give your first answer to that before reading further — don't skip ahead to the follow-ups yet. That first answer only earns you the first 20% of an interview score. The rest comes from what happens next.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

The point of this exercise isn't the first answer — it's proving you can anticipate the standing follow-up list yourself, *before* it's asked: **Why? Why not X? How? What happens underneath? What happens if it fails? How would you scale it? How would you cut its cost? How would you debug it?**

Give your first answer to the RAG-retriever scenario now, structured as a Normal-depth answer in this document's [Four-Depth Format](#) — one level past the 30-second claim, with enough method to show you'd actually know where to start. Then, before reading Hint 2, pick just **one** of the standing follow-ups and answer it against your own first answer. This is the actual senior-level habit: treating your own answer as something to interrogate, not something finished the moment you stop talking.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Now push one step further than the standing follow-up list: assume your answer to "what happens if it fails again" was some version of "I'd add a confidence threshold and have the system say it's not sure." A tough interviewer doesn't accept a fix at face value — they attack the fix itself: *"That threshold is going to make the system refuse to answer a lot of borderline-but-actually-fine queries. How do you know where to set it, and what does the product team say when 'I'm not sure' shows up for 15% of traffic?"*

This is a **follow-up on your own follow-up** — the interviewer is no longer testing whether you can debug the original scenario, they're testing whether your proposed fix was actually thought through, or just a plausible-sounding thing senior candidates are known to say. The skill here is different from Hint 1's: it's defending a design decision under direct challenge without either (a) collapsing and abandoning the fix, or (b) refusing to acknowledge the real cost the interviewer just named.

The real design question isn't "what's my fix" — it's "what would I actually say if someone with a legitimate point pushed back on my fix, and can I name the trade-off honestly instead of just repeating that the fix is a good idea?"

Try answering the pushback yourself before Hint 2.

Difference between Basic, Intermediate, and Advanced: Basic gives the scenario and lets you answer once, cold. Intermediate adds the standing follow-up list on top of that first answer — proving you can anticipate what comes next. Advanced goes one layer deeper still: a follow-up *on your follow-up*, which is what actually happens once an interviewer decides to dig into one specific part of your answer instead of moving down the checklist — the difference between reciting the standing follow-ups and actually defending a decision under real scrutiny.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The follow-up plan, worked through

Basic Version

Here's the plan written out in plain steps:

```
answer the scenario question first, plainly
```

```
then ask yourself, one at a time:
```

```
  what's actually happening underneath, in the retriever?
```

```
  what's the very first thing I'd check?
```

```
  is there a fallback if this happens again, or does it fail silently?
```

```
  (bonus) how would this get worse at 100x the traffic?
```

```
  (bonus) what's the cheap fix vs. the expensive fix?
```

```
answer each one in a sentence or two, out loud
```

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

The same plan, closer to interview structure, in the natural order for a debugging scenario:

```
first answer (Normal depth):
    state the general debugging approach for "worked in testing, broken in prod"

self-asked follow-ups, in a sensible order for THIS scenario:
    "what happens underneath?"
        -> name the specific layer: embeddings, chunking, or index staleness
    "how would you debug it?" (more specific than the first answer)
        -> name the first concrete check, and what result would confirm/rule out
    "what happens if it fails again?"
        -> is there a fallback path, or a silent bad answer?
    "how would you scale it / cut its cost?" (lighter touch, still answer it)
        -> one sentence: does the fix also affect cost or latency at scale?
```

Notice the order isn't random — it follows the natural shape of a debugging conversation: mechanism, then method, then resilience, then scale. Matching your follow-up order to the scenario's natural shape (instead of reciting the list in a fixed order every time) is itself a senior-level tell.

Answer at least three of these against your own first answer before checking the Solution.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Here's the plan for defending a fix under direct pushback:

```
your fix, as stated: "add a confidence threshold, return 'not sure'
                     below it instead of guessing"

interviewer pushback: "that'll refuse a lot of borderline-but-fine queries
                      – how do you pick the threshold, and what's the
                        product cost?"

your move:
do NOT abandon the fix just because a real cost was named
do NOT pretend the cost isn't real
name HOW you'd actually pick the threshold (not just that one exists):
    e.g. calibrate it against a labeled validation set, tune for a
          target false-refusal rate, not a guessed number
name the trade-off explicitly instead of dodging it:
    e.g. "yes, some fine queries get refused – that's the false-positive
          cost of ANY threshold, and it's tunable, not fixed"
connect it back to what happens WITHOUT the fix:
    a wrong-but-confident answer is worse than an honest refusal, for
    most products – but name which products that's NOT true for
```

Try filling this in yourself for the confidence-threshold pushback, then compare against the [Solution](#).

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)